

TDMA Schedule Planner and Optimizer

Technical Documentation — Part 1 Network Brain

Vaan Megam Networks — Wireless Protocol Development Internship
Assignment

Centralized Python-based simulator for Distance-2 TDMA scheduling in a wireless network.

Author	Nakul Joshi
Implementation	Python + NetworkX
Input	16-node coordinate topology (JSON)
Communication range	500 meters
Scheduling model	Distance-2 graph coloring
Current heuristic result	7 slots on the project test topology
Validation	Conflict-free
Automated tests	9 / 9 passed

Prepared as the project documentation deliverable. The document distinguishes implemented Part 1 functionality from the proposed Part 2 EMANE integration approach.

1. Executive Summary

The assignment asks for a centralized Python Schedule Optimizer for a TDMA network. Radios are modeled as graph nodes, wireless links are represented as graph edges, and radios within 500 meters are considered able to communicate. The scheduler must prevent both direct one-hop conflicts and two-hop receiver conflicts while permitting spatial reuse beyond two hops.

`filecite{turn1file2}{L1-L10}`

The implemented Part 1 solution builds a wireless connectivity graph with NetworkX, derives a Distance-2 conflict graph, evaluates multiple graph-coloring heuristics, validates the resulting schedule, and exports both a Node-to-Slot mapping and a Slot $\times$ Node Boolean matrix. The assignment explicitly permits heuristics because finding an absolute minimum coloring is NP-hard.

`filecite{turn1file2}{L10-L15}`

On the project's current 16-node test topology, the implementation creates 27 wireless links and 53 conflict links. It evaluates 103 coloring candidates and selects a valid 7-slot schedule. This 7-slot figure is reported as the heuristic result for this topology, not as a proof of the global minimum.

Automated tests cover graph construction, conflict generation, schedule validation, matrix generation, slot assignment, and invalid-input handling. The current test suite reports 9/9 passing tests.

Part 2, EMANE integration, is documented as an approach rather than claimed as complete. The assignment requests an EMANE TDMA Radio Model configuration and a bridge from the Python schedule to an XML or ProtoBuf schedule event; it also states that Part 1 is mandatory and that Part 2 may be incomplete. `filecite{turn1file3}{L1-L9}`

2. Assignment Requirements and Scope

The source assignment describes a TDMA scheduling problem in which radios share a frequency by taking turns in time slots. Nearby simultaneous transmissions can interfere, while sufficiently separated radios can reuse the same slot. [\[filecite\turn1file0\L1-L9\]](#)

The mandatory Network Brain requirements are:

- Use Python and NetworkX to represent radios as nodes and links as edges.
- Treat radios within 500 meters as communicating.
- Use Distance-2 graph coloring to prevent direct and two-hop conflicts.
- Permit spatial reuse when nodes are separated by more than two hops.
- Use heuristics to reduce the number of slots because exact minimum coloring is NP-hard.
- Provide a CLI that accepts a 16-node coordinate topology and produces a conflict-free Slot × Node Boolean matrix and Node-to-Slot mapping. [\[filecite\turn1file2\L10-L17\]](#)

The assignment's sample output uses a five-slot example topology. The implementation documented here uses a separate project test topology defined in `examples/nodes.json`, so its seven-slot result should not be interpreted as a reproduction of the sample's five-slot result. The source assignment explicitly presents the five-slot output as a sample. [\[filecite\turn1file1\L1-L8\]](#)

Part 2 asks for EMANE setup, native TDMA Radio Model XML profiles, and a bridge that converts the Python schedule matrix into an XML or ProtoBuf schedule event. [\[filecite\turn1file3\L1-L7\]](#)

3. System Architecture

The implementation is organized as a pipeline from topology input to validated schedule output.

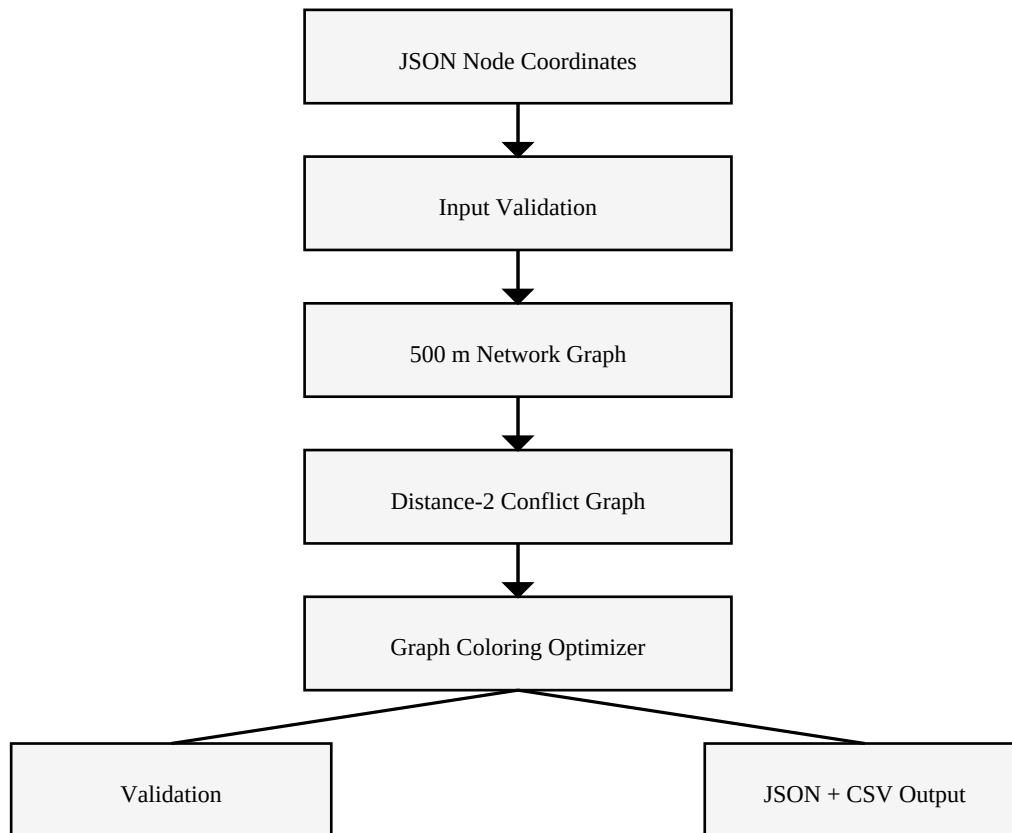


Figure 1. High-level architecture of the implemented Part 1 scheduler.

The main modules are separated by responsibility: `graph.py` handles physical connectivity, `scheduler.py` handles conflict modeling and slot optimization, `main.py` provides the CLI and output handling, and `visualize.py` produces a topology figure for analysis and presentation.

The resulting architecture keeps the mathematical scheduling logic independent from command-line input/output and visualization. This separation also makes the scheduler easier to test and extend toward a future EMANE integration layer.

4. Wireless Network Graph Model

Each radio is represented by a graph vertex. An undirected edge is created when the Euclidean distance between two radios is less than or equal to 500 meters, matching the assignment's communication assumption. [filecite{turn1file2}{L5-L9}](#)

For coordinates (x1, y1) and (x2, y2), the implementation uses the Euclidean distance:

$$d = \sqrt{(x2 - x1)^2 + (y2 - y1)^2}$$

The graph construction checks every unordered pair of nodes. For the current test topology, this produces 27 wireless links.

The topology is intentionally represented as an undirected graph because the assignment's connectivity model is based on whether radios can communicate within the configured range. The edge also stores the computed physical distance, which is useful for inspection and future extensions.

The current examples/nodes.json topology contains 16 nodes with coordinates spanning the test area. It is a project test topology rather than the exact coordinate set shown in the assignment's sample output.

5. Distance-2 Conflict Graph

Direct communication links alone do not capture the receiver-collision condition described in the assignment. The assignment requires nodes that are adjacent at one hop or share a common neighbor at two hops to use different slots. [filecite]turn1file2[L10-L15]

The implementation therefore constructs a second graph, called the conflict graph. For every pair of wireless nodes, the shortest-path distance in the connectivity graph is calculated. A conflict edge is added when that distance is 1 or 2.

Wireless graph relationship	Scheduling interpretation
1 hop	Direct interference / conflict
2 hops	Two-hop / hidden-terminal conflict
> 2 hops	Eligible for spatial slot reuse
Disconnected	No 1- or 2-hop path; no conflict edge from this rule

The resulting conflict graph for the current topology contains 53 conflict links. This graph is the actual input to the coloring stage.

6. TDMA Slot Optimization

The scheduling problem is mapped to graph coloring: each conflict-graph node represents a radio, each conflict edge means the two radios must have different colors, and each color represents a TDMA slot.

The optimizer evaluates several greedy coloring strategies rather than depending on a single node ordering:

- Largest-first greedy coloring.
- Smallest-last greedy coloring.
- DSATUR / saturation-largest-first coloring.
- 100 deterministic randomized greedy colorings using fixed seeds.

A candidate's cost is the number of distinct slot indices used. The optimizer retains the candidate with the smallest slot count. The fixed seeds make the randomized candidates reproducible across runs.

The source assignment explicitly states that the absolute minimum number of slots is NP-hard to find and asks for heuristics to reduce slot count while enabling spatial reuse.

`filecite{turn1file2L13-L15}`

For the current topology, 103 candidates are evaluated. The selected candidate is currently the largest-first coloring and uses seven slots. This is a heuristic result; the implementation does not claim that seven is the mathematical chromatic number of the conflict graph.

7. Schedule Validation and Output Representation

After coloring, every edge in the conflict graph is checked. If the two endpoints of a conflict edge have the same slot, the schedule is invalid and the conflicting pair is reported. A valid schedule has no such pair.

```
for (node_a, node_b) in conflict_graph.edges():
    assert slot[node_a] != slot[node_b]
```

The implementation generates two complementary output forms.

Output	Purpose
Node → Slot mapping	Direct representation of each radio's assigned TDMA slot.
Slot × Node matrix	Binary schedule representation where 1 means the node transmits in that slot.
schedule.json	Machine-readable schedule and validation summary.
schedule.csv	Tabular Slot × Node matrix for inspection or downstream processing.

For the current run, the seven slots are distributed as follows:

Slot	Nodes
0	Node_01, Node_03, Node_11
1	Node_02, Node_05, Node_07
2	Node_06, Node_08
3	Node_04, Node_10, Node_16
4	Node_09, Node_12
5	Node_14
6	Node_13, Node_15

8. CLI and Input Validation

The optimizer is exposed as a command-line application rather than being tied to one hard-coded topology.

```
.\.venv\Scripts\python.exe src/main.py --input examples/nodes.json
```

An alternate output directory can be supplied with the `--output-dir` option.

```
.\.venv\Scripts\python.exe src/main.py --input examples/nodes.json --output-dir outputs
```

The current assignment-focused input validator checks that the JSON root is an object, that exactly 16 nodes are supplied, that each node has two coordinates, and that coordinate values are numeric.

This validation is tested automatically so malformed topologies fail explicitly rather than producing an ambiguous scheduling result.

9. Testing

The project uses pytest for automated regression testing. The current suite contains nine tests.

Test area	What is verified
Network graph	Graph creation, node count, and 500 m edge behavior.
Conflict graph	Wireless links appear as conflicts.
Schedule validation	Generated coloring contains no conflict edge with equal slots.
Schedule matrix	Expected 0/1 matrix representation.
Node assignment	Every node receives a slot.
Slot range	Assigned slots are non-negative.
Node-count validation	Inputs other than exactly 16 nodes are rejected.
Coordinate validation	Malformed and non-numeric coordinates are rejected.

Latest local test result:

collected 9 items

9 passed

The tests are designed to catch regressions in both the scheduling logic and the CLI's input assumptions.

10. Complexity Analysis

Let n be the number of radios and m the number of wireless links.

Graph construction examines every unordered pair of radios, giving $O(n^2)$ distance checks.

The current conflict-graph implementation evaluates shortest-path distance for node pairs using NetworkX. The practical cost depends on graph density and the shortest-path implementation used by the library; for the assignment's 16-node topology this is small and easy to execute locally.

Greedy coloring itself is polynomial and is practical for the intended small topology. The optimizer deliberately runs multiple candidates to search for a lower-slot coloring rather than relying on one ordering.

The overall design prioritizes correctness and transparent heuristics for a small centralized network rather than attempting an exact exponential coloring solver.

The most important engineering trade-off is therefore between optimization effort and schedule quality: additional candidate colorings can improve the result, but they increase computation time.

11. Network Visualization and Results

A separate visualization module renders the physical topology using the same coordinates and schedule generated by the optimizer. Wireless links are shown between radios within the 500 m range, while node colors identify the assigned TDMA slot.

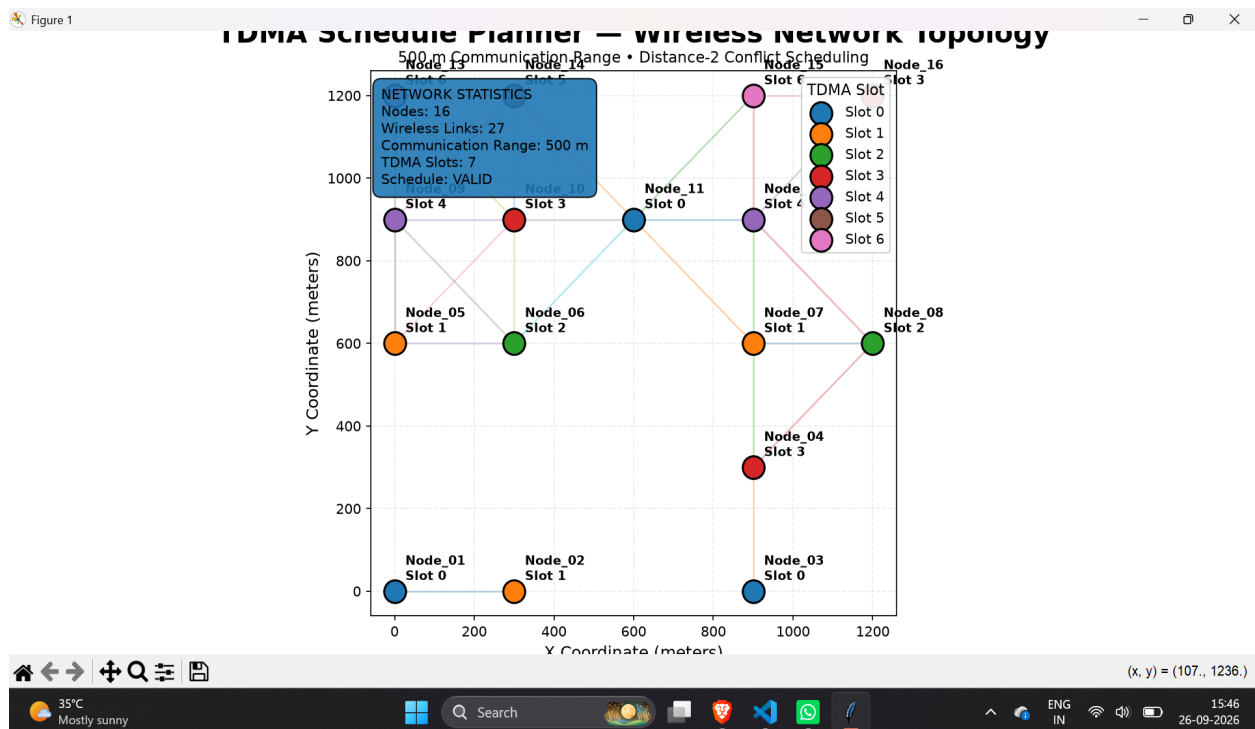


Figure 2. Generated topology visualization showing node positions, wireless links, and TDMA slot assignments.

The visualization is intended as an analysis and presentation aid. It shows the physical connectivity graph, not all 53 edges of the Distance-2 conflict graph, because rendering every conflict relationship would make the topology substantially harder to read.

The current result contains 16 nodes, 27 wireless links, seven assigned slots, and a conflict-free validation status.

12. Part 2 — EMANE Integration Approach

Part 2 is treated as an integration plan rather than a claim of completed implementation. The assignment asks for a lightweight local EMANE simulation, configuration of EMANE's native TDMA Radio Model XML profiles, and a bridge that converts the Python schedule matrix into an XML or ProtoBuf schedule event. [\[filecite\]turn1file3\[L1-L7\]](#)

Proposed integration pipeline:

```
Python topology
↓
TDMA optimizer
↓
Slot × Node matrix
↓
Schedule adapter / bridge
↓
EMANE TDMA schedule event
↓
EMANE virtual radios
↓
Packet transmission according to planned slots
```

The first engineering task would be to map the internal slot representation to EMANE's TDMA Radio Model concepts, including slot duration, frame structure, frequencies, and node-specific transmission opportunities. The assignment specifically references 1 ms slots, frames, frequencies, and an XML or ProtoBuf schedule event. [\[filecite\]turn1file1\[L8-L14\]](#)

A robust bridge should preserve node identifiers, slot indices, frame ordering, and any required radio/frequency metadata. The integration should then be verified by sending traffic between virtual nodes and observing that transmissions permitted by the schedule succeed while transmissions outside the planned schedule are prevented or dropped according to the configured EMANE model.

Because Part 2 is not implemented in the current project state, no EMANE packet-delivery result is claimed here.

13. Limitations and Engineering Considerations

- The current implementation is centralized; it assumes the complete topology is available to one optimizer.
- The communication model is based on a fixed 500 m Euclidean range, as required by the assignment, rather than a detailed RF propagation model.
- The scheduler uses graph-coloring heuristics. The selected seven-slot result is not presented as an exact minimum.
- The current conflict definition follows the assignment's one-hop/two-hop graph rule and does not model SINR, packet power, antenna patterns, fading, or channel capture.
- The input validator is intentionally assignment-specific and currently expects exactly 16 nodes.
- EMANE integration is planned but not claimed as completed.

These limitations are consistent with the assignment's focus on mathematical modeling, graph algorithms, scheduling logic, and a lightweight simulator rather than a full RF network emulator in Part 1. [filecite\[turn1file0\[L1-L9\]](#)

14. Project Structure

```
tdma-schedule-optimizer/  
  docs/  
  examples/  
  ■ nodes.json  
  outputs/  
  ■ schedule.json  
  ■ schedule.csv  
  ■ network_topology.png  
  src/  
  ■ graph.py  
  ■ scheduler.py  
  ■ main.py  
  ■ visualize.py  
  tests/  
  ■ test_scheduler.py  
  .gitignore  
  README.md  
  requirements.txt
```

Module responsibilities:

- graph.py — Euclidean distance calculation and 500 m wireless graph construction.
- scheduler.py — Distance-2 conflict graph, candidate colorings, heuristic selection, validation, and matrix generation.
- main.py — CLI parsing, input validation, schedule generation, and JSON/CSV export.
- visualize.py — topology and slot-assignment visualization.
- test_scheduler.py — automated regression and input-validation tests.

15. Conclusion

The implemented Part 1 solution provides the requested centralized TDMA scheduling brain. It converts a coordinate-based wireless topology into a NetworkX graph, transforms that graph into a Distance-2 conflict graph, applies multiple coloring heuristics, validates the result, and produces the requested schedule representations.

The current test topology produces 27 wireless links, 53 conflict links, and a valid seven-slot schedule after evaluation of 103 coloring candidates. The implementation also includes CLI input validation, JSON/CSV exports, automated tests, and a topology visualization.

The next engineering extension is the Part 2 bridge into EMANE. The documented approach is to translate the generated schedule into the event/configuration format expected by the EMANE TDMA Radio Model and then verify packet behavior in a virtual network.

The implementation deliberately separates verified results from future work: Part 1 is implemented and tested, while EMANE integration remains a planned extension.

16. References

1. Vaan Megam Networks, “Full-Time INTERNSHIP OFFER on Wireless Protocol Development,” assignment brief supplied for the internship task. The brief defines the 500 m connectivity model, Distance-2 coloring requirement, CLI deliverable, sample schedule, EMANE integration expectations, documentation requirement, and submission process. [\[filecite\turn1file0\L1-L19\]](#)
[\[filecite\turn1file2\L10-L17\]](#)
2. NetworkX documentation — used by the implementation for graph representation and graph-coloring utilities. The assignment itself recommends the NetworkX documentation/tutorial as a graph-theory resource. [\[filecite\turn1file3\L7-L9\]](#)
3. EMANE TDMA Radio Model — referenced by the assignment as the target simulation environment for Part 2. The assignment recommends reviewing the EMANE TDMA BDCE Model Guide for slots, frames, and multi-frequency schedules. [\[filecite\turn1file1\L8-L14\]](#)

Document status: Part 1 implementation and test documentation. Part 2 is an approach/design section and is not represented as completed functionality.